Capítulo 5. Pruebasautomatizadas

Este trabajo se ha traducido utilizando IA. Agradecemos tus opiniones y comentarios: translation-feedback@oreilly.com

> *La calidad no es un acto, es un hábito.*
>
> —Filósofo Will Durant, parafraseando a Aristóteles

Cuando comienza un nuevo año, muchos se fijan el objetivo de llevar y mantener un estilo de vida más saludable. Esos objetivos no se alcanzan comiendo una sola comida saludable o yendo al gimnasio una sola vez. Se trata de tomar decisiones saludables *de forma constante* la mayor parte del tiempo. Del mismo modo, producir software de alta calidad y fácil de mantener es el resultado de practicar *constantemente* buenos hábitos a la hora de escribir, revisar y probar tu código. Al igual que mantener un estilo de vida saludable, esto no es fácil, y las excusas solo te hacen retroceder.

Sin embargo, el camino hacia la constancia no siempre es fácil. Muchos desarrolladores luchan contra la inseguridad, preguntándose si están tomando las decisiones correctas o si su enfoque para resolver

problemas será cuestionado. Escribir software de alta calidad es una disciplina que requiere práctica regular, pero con el tiempo, tus esfuerzos de prueba servirán como salvaguardias efectivas que tus compañeros de equipo y tu yo futuro apreciarán. En este capítulo, aprenderás los beneficios de las pruebas automatizadas, los diferentes tipos de pruebas que encontrarás y cómo escribirlas.

Ventajas de las pruebas automatizadas

Es natural que te muestres escéptico ante la idea de escribir código adicional para verificar tu código existente, y a veces puede parecer que no vale la pena el esfuerzo. Quizás te preguntes si se trata solo de otra moda o de un ejercicio para mejorar tu currículum. Sin embargo, las pruebas automatizadas son mucho más que eso. Son una habilidad valiosa y una inversión importante en tu código base.

En las siguientes secciones, explorarás las ventajas concretas de escribir pruebas automatizadas. Examinarás cómo esta práctica puede mejorar la calidad de tu código, aumentar tu confianza en tu trabajo y, en última instancia, convertirte en un ingeniero más eficiente. Al comprender estas ventajas, obtendrás un enfoque claro para tus esfuerzos de prueba y apreciarás por qué invertir tiempo en pruebas automatizadas vale la pena para tu crecimiento profesional.

Actúa como documentación

Unirse a un proyecto existente puede resultar abrumador (consulta el capítulo 6 para saber qué hacer al unirte a un proyecto existente). Es un lujo que un proyecto cuente con documentación.[1] Si tienes suerte, quizá puedas consultar con los desarrolladores principales o con expertos en la materia para obtener una visión general de alto nivel antes de empezar a trabajar. Esto no siempre es posible, y la documentación puede ser escasa o incluso inexistente. Tu salvación es un proyecto que cuente con pruebas bien redactadas.

Imagina un escenario en el que te encarguen solucionar problemas de procesamiento de pagos con un tipo de tarjeta concreto. Sin la documentación o las pruebas adecuadas, tendrías que examinar código desconocido (probablemente complejo) o depender de tus compañeros para que te orienten.

Ahora imagina que encuentras un conjunto completo de pruebas para pagos. Verías una clase `PaymentProcesserTests` con nombres de pruebas descriptivos como estos:

- `shouldValidateValidCreditCardNumber`
- `shouldProcessCreditCardTransaction`
- `shouldProcessOrangePay`

Estas pruebas proporcionan información sobre las características del módulo y apuntan a secciones de código relevantes. Los IDE modernos permiten navegar fácilmente por los servicios específicos que se están probando (consulta el capítulo 2 para conocer estrategias eficaces de

lectura de código). Una prueba fallida te lleva al posible culpable dentro del código base.

Podrías descubrir una prueba llamada `shouldFailWhenCreditCardTypeIsOrangePay` , que revela que Orange Pay no es compatible. Esta información te ayuda a identificar y solucionar rápidamente el problema, actualizar la interfaz y mejorar la documentación.

Este ejemplo demuestra cómo las pruebas pueden servir como documentación, ayudándote a aprender más rápidamente un código base. Al centrarte en mejorar los conjuntos de pruebas, puedes mejorar la calidad del código, reducir los problemas de producción y acelerar tu capacidad para contribuir a los proyectos.

Mejora la mantenibilidad

Escribir buenas pruebas unitarias () y código fácil de mantener es una habilidad que suele llevar años desarrollar (consulta el capítulo 3 para obtener más información al respecto). Escribir primero las pruebas es como pensar antes de hablar: te ayuda a planificar y estructurar tus ideas. El objetivo principal de los principiantes suele ser simplemente hacer que el código funcione. Aunque esto sigue siendo importante, escribir pruebas te ayuda a reconocer mucho antes las áreas que se pueden mejorar.

`BlogPostController` Considera este ejemplo de un método (se ha omitido el código para abreviar):

```
public class BlogPostController {
   public void publish(Post post) {
       // save blog post to the database (flip isPublished to true)
       // log blog post has been published
       // send email to subscribers about the new post
   }
}
```

A primera vista, esto podría parecer perfectamente aceptable. Sin embargo, si intentas escribir pruebas para este método, pronto te darás cuenta de que hace demasiado:

- Comunicación con una base de datos
- Registro de información
- Envío de correos electrónicos

Este enfoque viola el principio de responsabilidad única, que establece: «Una clase solo debe tener una razón para cambiar».

Al pensar en cómo probar este código, identificas naturalmente sus defectos. A continuación, puedes refactorizar cada parte del proceso de publicación en clases separadas, lo que facilita su prueba y mantenimiento.

Escribir pruebas no solo verifica la funcionalidad, sino que también te guía hacia un mejor diseño del código. Te ayuda a detectar posibles problemas antes de que se conviertan en problemas, lo que da lugar a un software más fácil de mantener y reutilizar.

Aumenta tu confianza

Entre las muchas ventajas de las pruebas automatizadas, destaca una: la confianza para programar libremente. Como en muchos escenarios de la vida, proyectar confianza puede ayudarte a lidiar con la presión y a afrontar retos personales y profesionales. El desarrollo de software es intrínsecamente iterativo, ya que constantemente escribimos, experimentamos y refactorizamos. Sin pruebas, es posible que programes con cautela, tratando de evitar introducir cambios que rompan cosas. Es posible que tengas miedo de probar nuevas técnicas o soluciones creativas. Pero con un conjunto de pruebas robusto, se te proporciona una red de seguridad. Esta libertad cambia fundamentalmente tu forma de abordar la programación. Puedes realizar cambios, probar nuevas soluciones y refactorizar con confianza, sabiendo que tus pruebas detectarán cualquier problema.

Cambiar y refactorizar código puede ser una experiencia estresante. Es posible que no tengas un conocimiento completo de la aplicación y que los efectos secundarios sean difíciles de predecir. Trabajar sin pruebas es como escalar una montaña sin cuerdas. Si todo va bien, puede ser una descarga de adrenalina, pero un solo error puede ser catastrófico.

Las pruebas actúan como una red de seguridad al navegar por un código base y te permiten actuar con confianza. Si bien tu objetivo inicial es hacer que una función funcione, debes tener en cuenta cómo tus cambios pueden afectar a todo el sistema. Sin pruebas exhaustivas, todo puede parecer funcional y luego dar lugar a problemas durante la producción.

Un conjunto de pruebas sólido permite al equipo hacer lo siguiente:

- Reestructurar código con confianza
- Asegurar que los cambios no afecten a la funcionalidad existente
- Identificar posibles problemas antes de que lleguen a producción

Al escribir y mantener un conjunto completo de pruebas, podemos hacer lo siguiente:

- Comprender mejor el comportamiento del sistema
- Detectar errores en una fase temprana del proceso de desarrollo, cuando son más sencillos y económicos de solucionar
- Reduce la probabilidad de introducir regresiones
- Asegurarte de que las características cumplan con las especificaciones y mantener la calidad del código mediante pruebas automatizadas

Este enfoque no solo mejora la calidad del código, sino que también aumenta tu confianza como desarrollador. Puedes realizar cambios, añadir funciones y refactorizar con la seguridad de que tus pruebas detectarán posibles problemas.

La confianza en la codificación proviene de una combinación de conocimientos, experiencia y herramientas. Las pruebas automatizadas son un poderoso aliado para generar y mantener la confianza a lo largo de tu carrera como desarrollador.

Conduce a la coherencia y la repetibilidad

En un momento dado, los desarrolladores solían depender de innumerables horas de pruebas manuales. Se creaba una lista de pasos para poner a prueba la nueva función. Aunque se siguiera el guion la mayor parte del tiempo, el enfoque manual es propenso a errores, no es repetible y requiere mucho trabajo y tiempo. Los seres humanos son inherentemente incapaces de realizar la misma tarea repetidamente sin variaciones.

Las pruebas automatizadas ofrecen resultados consistentes y repetibles. A diferencia de los seres humanos, los guiones de prueba siguen exactamente los mismos pasos cada vez, sin errores ni omisiones. Estas pruebas se ejecutan rápidamente con un mínimo esfuerzo, con solo hacer clic en un botón o durante las compilaciones. Las pruebas de regresión verifican específicamente que la funcionalidad existente siga funcionando a medida que cambia el código.

Con un conjunto completo de pruebas, puedes asegurarte de que se ejecuten de forma coherente y repetida, independientemente del entorno. Las pruebas automatizadas se pueden ejecutar con frecuencia,[2] lo que proporciona pruebas de regresión eficaces. Estas pruebas garantizan que los errores corregidos anteriormente no vuelvan a aparecer misteriosamente; las pruebas también identifican nuevos errores introducidos por los cambios en el código base. Cuando pruebes una nueva función, omite las costosas pruebas manuales y confía en las pruebas automatizadas.

Tipos de pruebas automatizadas

Ahora que sabes por qué debes realizar pruebas, exploremos qué debes (¡y qué no debes!) probar. *Las pruebas automatizadas* son una categoría amplia que abarca pruebas de interfaz de usuario, pruebas de extremo a extremo y pruebas de integración, así como pruebas unitarias. Es común pensar en estos tipos de pruebas como una pirámide, como se muestra en la Figura 5-1.

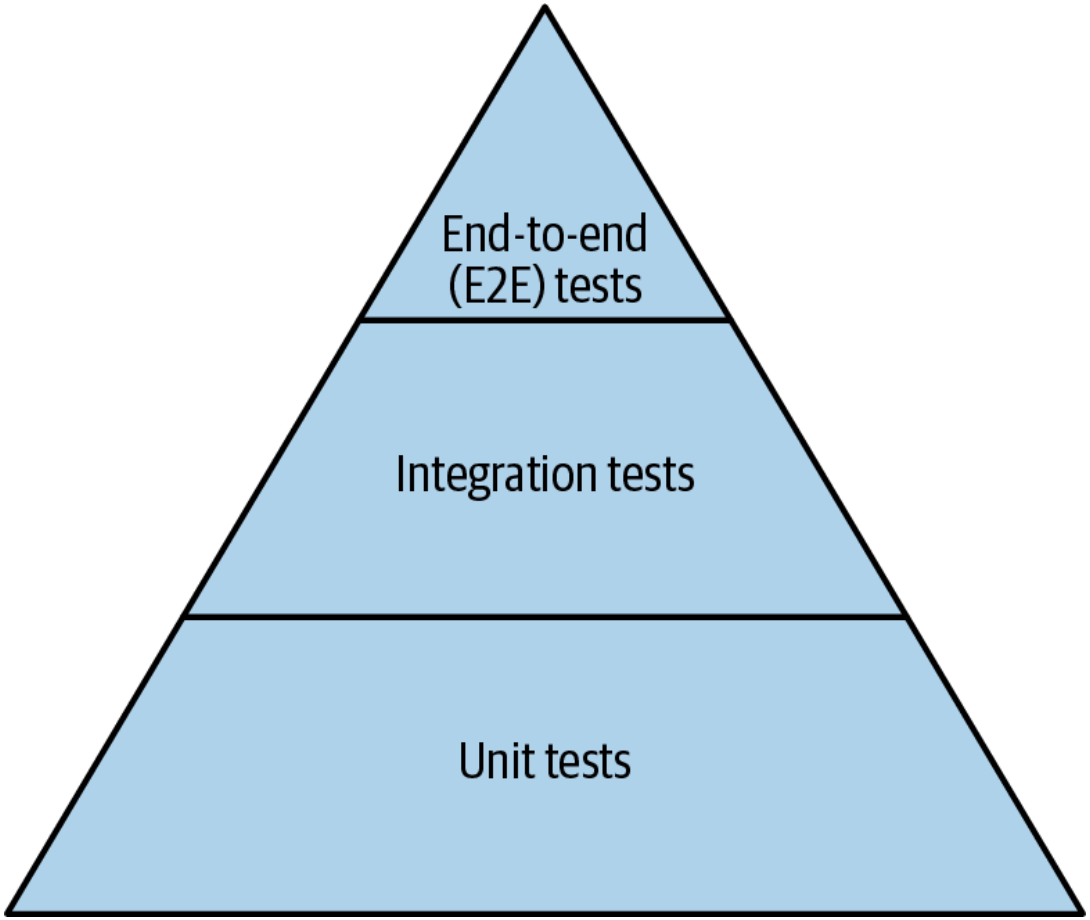


Figura 5-1. Pirámide que muestra los tres tipos de pruebas automatizadas que tendrás que escribir y la cantidad recomendada de cada tipo en relación con los demás

La pirámide de pruebas es un concepto importante en las pruebas de software, pero a menudo se malinterpreta o se ignora por completo. Creado por Mike Cohn, este modelo proporciona una guía visual de los tipos de pruebas que debes realizar en tu aplicación.

Estos son los tres tipos de pruebas automatizadas:

Pruebas unitarias

Diseñadas para cubrir componentes individuales de funciones aisladas del resto del sistema, lo que garantiza que cada parte funcione correctamente por sí sola.

Pruebas de integración

Verifican cómo los diferentes componentes o módulos de un sistema funcionan juntos como una unidad cohesionada.

Pruebas de extremo a extremo

Cubren toda la aplicación, desde la interfaz de usuario hasta el sistema backend.

Esta estructura se basa en compensaciones fundamentales en las pruebas de software. A medida que se asciende en la pirámide, las pruebas se vuelven más lentas de ejecutar, más caras de mantener y más propensas a fallar. Una prueba unitaria puede ejecutarse en microsegundos, mientras que una prueba de extremo a extremo puede tardar varios minutos. Cuando se tienen cientos o miles de pruebas, estas diferencias se acumulan de forma espectacular. Veamos más de cerca cada una de ellas.

Pruebas unitarias

Las pruebas unitarias constituyen la base de la pirámide de pruebas y representan la mayor parte de tu conjunto de pruebas. Piensa en ellas como un contrato que tu código debe cumplir. Si el código cambia de alguna manera funcional, las pruebas unitarias correspondientes deberían fallar, alertándote de posibles problemas. Estas pruebas examinan componentes o funciones individuales de forma aislada, asegurando que cada pieza funcione correctamente por sí sola.

Las pruebas unitarias sirven como primera línea de defensa contra los defectos, junto con el análisis estático y las revisiones de código. Deben ser rápidas de escribir y ejecutar, y proporcionar una respuesta rápida durante el desarrollo. Cuando una prueba unitaria falla, normalmente apunta a una función o línea de código específica, lo que facilita la depuración. Esta rapidez y precisión hacen que las pruebas unitarias sean muy valiosas durante el desarrollo de funciones y la integración continua, ya que dan a los desarrolladores la confianza necesaria para iterar y mejorar su código rápidamente.

Pruebas de integración

Una *prueba de integración* es un proceso detallado que verifica minuciosamente cómo los diferentes componentes o módulos de un sistema funcionan juntos como una unidad cohesionada. Mientras que las pruebas unitarias se centran en partes individuales del código de forma aislada, las pruebas de integración abarcan un ámbito más amplio al examinar las interacciones entre estas partes.

Sin embargo, siguen teniendo un enfoque más limitado en comparación con las pruebas completas del sistema, que evalúan el rendimiento y la funcionalidad de todo el sistema. Las pruebas de integración son cruciales porque ayudan a detectar problemas que solo surgen cuando se combinan componentes probados individualmente, lo que garantiza que el sistema integrado funcione de forma correcta y eficiente. Este paso es esencial en el SDLC para mantener la integridad y la fiabilidad del sistema en su conjunto.

Debido a su alcance y complejidad, las pruebas de integración suelen tardar más en ejecutarse que las pruebas unitarias y, a menudo, se ejecutan con menos frecuencia. Cuando las pruebas de integración fallan, normalmente es necesario investigar más a fondo para identificar qué interacción entre los módulos ha causado el problema.

Pruebas de extremo a extremo

Una *prueba de extremo a extremo (E2E)* es un procedimiento de prueba exhaustivo que abarca toda la aplicación, desde la interfaz de usuario hasta los sistemas backend. Estas pruebas están diseñadas para simular escenarios de usuarios reales, lo que garantiza que la aplicación funcione como se espera en un entorno real.

Debido a su naturaleza compleja y exhaustiva, estas pruebas suelen ser lentas de ejecutar y requieren importantes recursos y tiempo para llevarse a cabo. Como resultado, por lo general encontrarás menos pruebas de extremo a extremo en comparación con otros tipos. A menudo se ejecutan con menos frecuencia, normalmente como parte de

un proceso programado. Las pruebas de extremo a extremo pueden ser frágiles y son más susceptibles a falsos negativos (es posible que una prueba falle debido a un cambio trivial en un elemento de la interfaz de usuario, en lugar de a un problema real con la funcionalidad). El fallo de las pruebas de extremo a extremo puede requerir un esfuerzo considerable de depuración para identificar y rectificar el problema.

A pesar de ello, las pruebas de extremo a extremo son muy valiosas, ya que proporcionan un alto grado de confianza en la funcionalidad general del sistema y ayudan a garantizar que todos los componentes de la aplicación funcionen juntos a la perfección.

¿Qué combinación de pruebas debes escribir?

El número real de pruebas variará de un proyecto a otro, dependiendo de la complejidad de la aplicación, la pila tecnológica utilizada y la criticidad de la aplicación. Además, el número de pruebas podría basarse en las normas y prácticas de tu organización, que pueden incluir directrices o puntos de referencia específicos para la cobertura y la calidad de las pruebas. Es importante adaptar tu estrategia de pruebas a las necesidades específicas de tu proyecto, al tiempo que se garantiza un enfoque equilibrado que cubra todos los aspectos necesarios de la aplicación.

Lo que no debes probar

Si bien las pruebas exhaustivas son esenciales para la calidad del software, es igualmente importante ser estratégico en cuanto a lo que se prueba. A continuación se presentan las directrices clave sobre lo que debes evitar en tu estrategia de pruebas:

- Debes centrarte en tu código y no en probar las características del lenguaje o el código del marco.[3]
- Evita probar código generado como getters, setters, métodos de construcción y objetos de transferencia de datos (DTO) generados automáticamente.
- Evita probar métodos privados directamente. En su lugar, céntrate en la interfaz pública que llama a estos métodos privados.
- Evita escribir pruebas que dependan de servicios externos. En su lugar, utiliza simulaciones para las pruebas unitarias o dobles de prueba para las pruebas de integración.

Un conjunto de pruebas bien diseñado debe ser completo pero fácil de mantener, y proporcionar una cobertura exhaustiva sin resultar difícil de manejar. Si te centras en probar tu propio código y evitas pruebas innecesarias, podrás crear conjuntos de pruebas más eficaces y eficientes. Ahora que ya conoces los tipos de pruebas automatizadas que existen y en qué debes centrar tus esfuerzos, es el momento de hablar de la cobertura del código.

Cobertura del código

La cobertura del código es una métrica que se utiliza para medir el porcentaje de código que se ejecuta cuando se realizan las pruebas. Piensa en la cobertura del código como un mapa en un videojuego. Cuando empiezas, todo el mapa está oscuro, pero a medida que te mueves y exploras las zonas, estas se van haciendo visibles. Las pruebas que ejecutan partes de tu código «iluminan» esas secciones, mostrando que has estado allí. Una cobertura alta significa que has explorado la mayor parte del mapa, mientras que una cobertura baja significa que hay puntos ciegos donde pueden esconderse errores.

Dada la importancia de la cobertura de código en el mundo de las pruebas, encontrarás una gran variedad de IDE que admiten la cobertura de código, así como herramientas para el lenguaje que elijas. Como desarrollador, puedes ejecutar la cobertura directamente en tu IDE para obtener información instantánea sobre tu cobertura. También hay herramientas que se integran en tu canalización de CI/CD para aplicar un umbral mínimo de cobertura de código. Si la cobertura cae por debajo de ese umbral, pueden fallar, lo que a su vez bloquea la fusión de una PR.

Las herramientas de cobertura de código analizan tus ejecuciones de pruebas y generan informes que muestran exactamente qué líneas, ramas y funciones se ejecutaron. Es posible que veas porcentajes como «85 % de cobertura de línea» o «72 % de cobertura de rama», que indican qué parte de tu código se ejecutó durante las pruebas.

Aunque la cobertura del código proporciona información valiosa, es importante comprender sus limitaciones. Un error común es pensar que una mayor cobertura del código significa automáticamente mejores pruebas. Algunas organizaciones incluso exigen umbrales mínimos de cobertura del 80 % al 90 % antes de poder realizar la implementación. Aunque estos requisitos tienen buenas intenciones, también pueden dar lugar a comportamientos contraproducentes.

Como desarrollador, puedes escribir pruebas que ejecuten código sin verificar realmente un comportamiento significativo, esencialmente «jugando» con las métricas de cobertura. Una prueba que llama a un método pero no afirma nada útil sigue contando para la cobertura, pero no proporciona una protección real contra los errores.

En lugar de esforzarte por alcanzar una cobertura del 100 %, que a menudo es una métrica vanidosa, utilízala como mecanismo de retroalimentación para las pruebas que estás escribiendo. Una baja cobertura en la lógica empresarial crítica para la misión podría indicar un área del código que necesita más atención. Una alta cobertura en áreas en las que simplemente estás probando clases de portadores de datos o configuraciones podría sugerir que estás probando código que no lo requiere.

Céntrate en escribir pruebas significativas que verifiquen el comportamiento y deja que la cobertura del código te guíe hacia las áreas de tu código que necesitan más atención. El objetivo no es alcanzar una cifra arbitraria, sino cumplir un requisito. El objetivo de las pruebas es generar confianza en el código que estás escribiendo y que, en última instancia, se enviará a producción. Ahora que entiendes la cobertura

del código, empecemos a escribir algunas pruebas para que puedas empezar a utilizar estas herramientas.

Escribir pruebas

Es hora de ponerse manos a la obra y aprender la mecánica de escribir pruebas.

Introducción

Independientemente del lenguaje de programación, el marco o el metamarco que utilices, es probable que tengas a tu disposición múltiples herramientas de pruebas. En esta sección, vamos a utilizar el lenguaje de programación Java. Aunque la configuración y la sintaxis pueden ser diferentes para el lenguaje que elijas, las ideas siguen siendo las mismas.

Una vez que hayas creado un proyecto, tu siguiente decisión será el marco de pruebas que vas a utilizar. Hay muchas opciones excelentes entre las que elegir, pero el marco de pruebas más popular para Java se llama JUnit. Para incluir JUnit, puedes declarar las dependencias adecuadas en tu archivo Maven POM e instalarlas.

No vamos a entrar en detalles sobre JUnit aquí, pero si te interesa saber más al respecto, te recomendamos que consultes la excelente documentación de su guía de usuario. Una vez que tengas JUnit instalado, puedes empezar a escribir pruebas.

Hay dos enfoques principales para escribir pruebas: prueba primero y prueba al final:

- *Las metodologías de prueba primero*, como el desarrollo basado en pruebas (TDD), siguen un ciclo «rojo-verde-refactorizar» . Primero, escribes una prueba que falla («rojo»), luego implementas el código mínimo para pasar la prueba («verde») y, finalmente, mejoras el código sin cambiar su comportamiento («refactorizar»).
- *Los enfoques de prueba posterior* implican escribir pruebas después de implementar la funcionalidad.

Ambos métodos tienen sus ventajas, y la decisión entre uno u otro suele depender de los requisitos del proyecto y las preferencias del equipo. Independientemente del enfoque, todas las metodologías de prueba utilizan aserciones para verificar los resultados esperados.

NOTA SOBRE IA

Escribir pruebas es un excelente caso de uso para la asistencia de IA. Puedes proporcionar a las herramientas de IA ejemplos de tu conjunto de pruebas existente para ayudarlas a comprender el estilo de prueba, las convenciones de nomenclatura y los estándares de codificación preferidos de tu organización. La IA puede ayudar a generar casos de prueba, sugerir casos de perímetro que podrías haber pasado por alto, escribir código de prueba repetitivo e incluso ayudar a refactorizar las pruebas existentes para mejorar la legibilidad. Esto puede acelerar significativamente el proceso de prueba mientras se mantiene la coherencia con los patrones establecidos por tu equipo.

Aserciones

Una*aserción de JUnit ( )* es una función o método que se puede utilizar para verificar que se cumplen determinadas condiciones durante la ejecución de un programa. Existen varias bibliotecas de aserciones en Java, todas ellas con diferentes características.

JUnit 5 incluye varias aserciones para realizar una amplia gama de verificaciones. Todas las aserciones son métodos estáticos de la clase `` `org.junit.jupiter.Assertions` ``. Supongamos que quieres verificar que la suma de una operación matemática es correcta. Existe un método `` `assertEquals` `` que toma dos enteros, como se muestra en el siguiente bloque de código. El primer argumento es el valor esperado y el segundo es el valor real:

```
@Test
void shouldAdd2Numbers() {
   int expected = 3;
   int actual = 2 + 1;
   assertEquals(expected,actual);
}
```

Si los dos números son iguales, la aserción se supera y, por lo tanto, también la prueba. Si no son iguales, la aserción falla y provoca que la prueba falle. Para familiarizarte con las aserciones disponibles, lee la documentación de la biblioteca de aserciones que estés utilizando.

Escribir pruebas unitarias

Recuerda que las pruebas unitarias de son pruebas aisladas que se ejecutan de forma independiente y muy rápida. Imagina una clase que realiza operaciones matemáticas que requieren pruebas para cada una de ellas. Aquí tienes un ejemplo de una clase llamada `Operations`:

```
package com.fose;

public class Operations {
    public double add(double a, double b) {
        return a + b;
    }
    public double subtract(double a, double b) {
        return a - b;
    }
}
```

Antes de crear tu primera prueba, debes prestar atención a la ubicación del código fuente. El código de esta clase se encuentra en el paquete `com.fose`. Cuando crees una clase de prueba, debes colocarla en */src/test/java* y en el mismo paquete en el que se encuentra el código fuente. En el siguiente ejemplo, tenemos una prueba de suma y resta para cubrir todas las funciones de la clase `Operations`.

Aquí estás siguiendo enfoques de prueba final, que implican escribir pruebas después de implementar la funcionalidad. Puedes comenzar

agregando los stubs de método para cada una de las pruebas que vas a escribir:

```
package com.fose;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class OperationsTest {
   @Test
   void add() {
   }

   @Test
   void subtract() {
   }
}
```

En primer lugar, necesitas una instancia de la clase `Operations` . A menudo oirás hablar de esto como el*sistema bajo prueba* (*SUT*) , ya que se refiere al componente, módulo o parte específica de tu sistema que estás probando actualmente. Se trata de una prueba unitaria que se centra en probar la clase `Operations` y nada más:

```
class OperationsTest {

   private final Operations sut = new Operations();
}
```

Con una instancia del sistema bajo prueba disponible, ahora puedes completar los métodos de prueba. No estamos haciendo nada especial aquí, pero debes asegurarte de cubrir todos los casos de perímetro, ya que no quieres que tus usuarios los detecten por ti:

```
package com.fose;

import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class OperationsTest {
   private final Operations sut = new Operations();
   @Test
   void add() {
       assertEquals(5, sut.add(2,3));
       assertEquals(0, sut.add(-1,1));
       assertEquals(-5, sut.add(-2,-3));
   }
   // Additional tests omitted for brevity
}
```

Simulación

En las pruebas de software, *la simulación* es una técnica que se utiliza para crear una versión simulada de una dependencia (a menudo denominada *«doble»* en este contexto) que permite probar el código de forma aislada.

¿Qué es una dependencia? En el contexto de la escritura de una prueba unitaria, una *dependencia* es cualquier otra clase de la que depende el sistema sometido a prueba para completar una tarea. Si bien la simulación ayuda a aislar las dependencias en las pruebas unitarias, presenta un desafío importante en las pruebas de integración. En las pruebas de integración, el objetivo es verificar cómo funcionan juntas estas dependencias, lo que puede resultar complicado. Gestionar múltiples dependencias, asegurarte de que estén en el estado correcto y manejar sus interacciones puede complicar rápidamente tus pruebas de integración. Es como intentar hacer malabares con varias pelotas a la vez: necesitas llevar un registro de cómo se comporta cada dependencia y cómo afecta a las demás.

El siguiente ejemplo tiene una clase llamada `UserService` con dos dependencias: `UserRepository` y `EmailService`. La clase `UserRepository` se encarga de leer y conservar los usuarios en tu base de datos, mientras que `EmailService` se encarga de enviar correos electrónicos:

```
package com.fose;

public class UserService {

    private UserRepository userRepository;
    private EmailService emailService;

    public UserService(UserRepository userRepository, EmailService emailService) {
        this.userRepository = userRepository;
        this.emailService = emailService;
    }
```

```
    public void registerUser(String username, String email) {
        // register user
    }
}
```

Si quieres escribir una prueba unitaria para la clase `UserService` que se centre en probar el método `registerUser`, ¿cómo lo harías? En este momento, esta clase tiene dos dependencias, y si incluyes ambas, ya no estarás escribiendo una prueba unitaria, sino una prueba de integración.

Aquí es donde entra en juego un marco de simulación. En lugar de utilizar la clase `UserRepository` de producción, utilizarás una simulación de esa clase; la simulación *se parece* a la clase, pero no lo es. `UserRepository` Probablemente hayas oído la frase «actúa como un pato, grazna como un pato, pero no es un pato». Esta simulación se parecerá a una clase, pero no lo es y no realizará la lógica de negocio *real*, como persistir un nuevo usuario en una base de datos. En su lugar, hará lo que tú programes que haga la simulación.

Céntrate solo en probar tu clase de simulación (`UserService`); las dependencias se probarán en sus propias pruebas unitarias. En este ejemplo, utilizamos un popular marco de simulación para el mundo Java llamado Mockito, pero, de nuevo, los conceptos deberían poder trasladarse a cualquier lenguaje que estés utilizando.

Con Mockito, puedes escribir un `UserServiceTest` :

```
@ExtendWith(MockitoExtension.class)
class UserServiceTest {

    @Mock
    private UserRepository userRepository;
    @Mock
    private EmailService emailService;
    @InjectMocks
    private UserService userService;

    @Test
    public void testUserRegistration() {
        String username = "newuser";
        String email = "newuser@example.com";
    when(userRepository.existsByUsername(username)).thenReturn(false);
        userService.registerUser(username, email);
        verify(userRepository).save(any(User.class));
        verify(emailService).sendWelcomeEmail(email);
    }

}
```

En el código de prueba anterior, los objetos simulados se crean utilizando la anotación `@Mock` para simular dependencias, lo que permite realizar pruebas controladas sin necesidad de implementaciones reales de. A continuación, estas dependencias simuladas se inyectan automáticamente en la clase que se está probando.

Escribir pruebas de integración

En la sección anterior, viste un ejemplo con `UserService`, que dependía tanto de `UserRepository` como de `EmailService`. Al escribir una prueba unitaria, querías aislar esa clase y, por lo tanto, simulaste las dos dependencias.

En una prueba de integración, quieres probar cómo interactúa `UserService` con sus dependencias. Empieza creando una nueva prueba llamada `UserServiceIntTest`. El `Int` como parte del nombre identifica claramente esta prueba como una prueba de integración.

`UserService` depende tanto del repositorio como del servicio de correo electrónico, por lo que necesitarás instancias de cada una de esas clases para crear una instancia de `UserService`. En una aplicación real, es posible que utilices un marco con inyección de dependencias, lo que te permite solicitar una instancia, pero aquí tendrás que crearlas manualmente:

```
class UserServiceIntTest {
   private UserRepository userRepository;
   private EmailService emailService;
   private UserService userService;

   @BeforeEach
   void setUp() {
       userRepository = new UserRepository();
       emailService = new EmailService();
       userService = new UserService(userRepository,emailService);
```

```
    }

}
```

Ahora que tienes una instancia de `UserService` que contiene versiones reales (no simuladas) de tus dependencias, puedes escribir algunas pruebas de integración:

```
@Test
void shouldNotRegisterUserWithUsernameOfUser() {
    // Test valid user registration
}

@Test
void shouldRegisterValidUser() {
    // Test invalid user registration
}
```

Escribir pruebas de extremo a extremo

Las pruebas E2E se centran en la API backend. Estas pruebas requieren un entorno real para ejecutarse, en este caso, Spring Boot con un servidor Tomcat integrado. El objetivo es garantizar que un servidor real esté funcionando, enviando solicitudes y validando respuestas. Aquí hay un ejemplo conciso de una prueba E2E (el ejemplo completo se puede encontrar en GitHub):

```
@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)
@AutoConfigureMockMvc
```

```
class UserRegistrationE2ETest {

    @Autowired private MockMvc mockMvc;
    @Autowired private UserRepository userRepository;
    @Autowired private EmailService emailService;
    @Autowired private ObjectMapper objectMapper;

    @BeforeEach
    void setUp() {
        userRepository.deleteAll();
    }

    @Test
    void testUserRegistration() throws Exception {
        String username =
                "testuser";
        String email = "testuser@example.com";
        User request = new User(username, email);

        // Perform the request
        mockMvc.perform(post("/api/users/register")
                .contentType(MediaType.APPLICATION_JSON)
                .content(objectMapper.writeValueAsString(request)))
                .andExpect(status().isOk());

        // Verify that the user was saved in the database
        Optional<User> savedUser = userRepository.findByUsername(username);
        assertTrue(savedUser.isPresent());
        assertEquals(email, savedUser.get().email());

        // // Additional verifications (welcome email sent) omitted for brevity
    }
}
```

Esta prueba hace bastante:

1. Configura un puerto aleatorio para el servidor
2. Utiliza `MockMvc` para la compatibilidad con las pruebas Spring MVC del lado del servidor
3. Limpia la base de datos antes de cada prueba
4. Simula una solicitud de registro de usuario
5. Verifica la respuesta y el estado de la base de datos

Este enfoque te permite probar todo el flujo, desde la solicitud de la API hasta la interacción con la base de datos, lo que garantiza que tu sistema funcione correctamente de principio a fin.

Conclusión

A lo largo de este capítulo, has explorado el mundo de las pruebas automatizadas y su papel crucial en el desarrollo de software. Has visto cómo las pruebas sirven como documentación valiosa, mejoran la calidad del código y actúan como red de seguridad para la refactorización. Desde las pruebas unitarias hasta las pruebas de extremo a extremo, cada tipo desempeña un papel fundamental para garantizar que tu software funcione según lo previsto.

Recuerda que, al igual que mantener un estilo de vida saludable, escribir buen software consiste en establecer hábitos positivos y tomar decisiones coherentes y saludables. Programar con un conjunto de pruebas robusto es como tener un entrenador y un plan de ejercicios bien definido. Las pruebas te dan la confianza necesaria para avanzar, pro-

bar cosas nuevas y realizar cambios sin el temor constante de cometer errores. Esta confianza te permite navegar por las complejidades del desarrollo de software, lo que te permite mejorar continuamente y adaptarte a los nuevos requisitos sin temor a romper la funcionalidad existente.

A medida que avanzas en tu carrera, esfuérzate por convertir las pruebas en un hábito. Al principio puede parecer un trabajo extra, pero los beneficios a largo plazo para la calidad de tu código, la mantenibilidad de tus proyectos y tu crecimiento profesional son inconmensurables. Recuerda que la calidad no es algo puntual, sino una práctica constante. Al adoptar las pruebas automatizadas, no solo estás detectando errores, sino que te estás convirtiendo en un desarrollador más eficaz, seguro y cualificado.

Ponerlo en práctica

Las pruebas no son una habilidad que se perfecciona de la noche a la mañana, sino que se desarrolla a través de la práctica constante. Las siguientes prácticas ofrecen varios puntos de entrada al mundo de las pruebas automatizadas, tanto si estás trabajando en una nueva función, corrigiendo un error o mejorando una base de código. No te sientas abrumado por intentar implementar todas estas prácticas a la vez. En su lugar, elige una o dos prácticas que se adapten a tu situación actual y empieza por ahí. A medida que te sientas cómodo con estos hábitos, incorpora gradualmente otros. Recuerda que incluso escribir una sola prueba hoy es mejor que no escribir ninguna. Aquí tienes algunas prácticas que te ayudarán a desarrollar tus habilidades de prueba:

Aprende de las bibliotecas de código abierto.

Explora tu biblioteca de código abierto favorita:

- Lee las pruebas.
- ¿Qué has aprendido sobre la funcionalidad de la biblioteca que no sabías antes?
- ¿En qué se diferencian sus prácticas de prueba de las tuyas?
- Considera la posibilidad de adoptar algunas de sus estrategias de prueba en tus propios proyectos.

Utiliza las pruebas como documentación

Sigue estos pasos cuando te asignen trabajar en una parte desconocida del código base:

- Busca primero las pruebas existentes.
- Si faltan pruebas o son inadecuadas, escribe otras nuevas a medida que aprendas sobre el código.

Mejora los códigos base existentes.

Sigue estos pasos si descubres que tu proyecto tiene un número insuficiente de pruebas:

- No intentes arreglar todo de una vez. En su lugar, realiza mejoras incrementales.
- Desafía a cada desarrollador (incluido tú mismo) a escribir una nueva prueba al día.

Empieza con pruebas para las nuevas funciones

La próxima vez que te asignen una nueva función, empieza por escribir pruebas. Esta práctica te ayudará a aclarar los requisitos y a diseñar un código mejor desde el principio.

Aborda los errores con pruebas

Cuando te enfrentes a un error, haz lo siguiente:

- Primero, escribe una prueba que verifique que el error existe. Esta prueba debería fallar inicialmente.
- A medida que escribes las pruebas, esto puede suponer una oportunidad para refactorizar el código con el fin de mejorar la calidad, la legibilidad o la mantenibilidad.

Abogar por las pruebas

Haz lo siguiente si encuentras un problema que se podría haber evitado con pruebas adecuadas:

- Añade pruebas que hubieran detectado el problema.
- Aprovecha esto como una oportunidad de aprendizaje para el equipo.
- Discute con tu gerente cómo una mejor cobertura de las pruebas podría evitar problemas similares en el futuro.

Práctica

La única forma de mejorar en la redacción de pruebas es practicar. Considera la posibilidad de practicar con katas de código

para desarrollar tus habilidades de prueba de forma estructurada e iterativa.

Recuerda que crear un conjunto de pruebas sólido es un proceso continuo. Cada pequeño paso que des hacia mejores prácticas de prueba contribuye a la calidad y la mantenibilidad generales de tu código base.

Recursos adicionales

- Principio de responsabilidad única
- Mockito
- JUnit 5
- AssertJ
- Hamcrest
- *Código limpio: un manual de artesanía ágil en software*, de Robert C. Martin (Pearson, 2008)

[1] Es aún más raro que la documentación esté actualizada y sea precisa.

[2] Probablemente en cada registro como parte de un proceso de integración continua.

[3] Los responsables del lenguaje y del marco son los encargados de probar su código, no tú.